Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Programmation Parallèle — TP3 — Hands-on CUDA – Hello World et Addition de Vecteurs sur GPU
4e année Cybersécurité - École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
KABORE Mohammad Sharif Jonathan

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. BAKHOUYA

Date : 16 octobre 2026

Table des matières

Rappel des objectifs du TP **2**

Rappels sur le modèle d'exécution CUDA **2**

Architecture du projet **4**

Protocole de mesure et validation **4**

Complexité théorique 5

1 Exercice 0 : CUDA Hello World **6**

2 Exercice 1 : Addition de vecteurs **7**

2.1 Principe 7

2.2 Résultats 9

2.3 Analyse complémentaire 12

3 Synthèse et interprétation **13**

Conclusion **14**

Rappel des objectifs du TP

Ce premier lab a pour objectif de découvrir la programmation GPU avec **CUDA** : écrire, compiler et lancer un premier noyau, puis porter une opération simple sur GPU et **analyser l'influence de la configuration de la grille** sur les performances. Deux exercices sont réalisés :

- **Exercice 0 : CUDA Hello World** — compilation avec `nvcc` d'un programme C pur (`cuda_ex_0.c`), puis ajout d'un noyau vide marqué `__global__` et lancé avec `<<<1, 1>>>` (`cuda_ex_0.cu`)
- **Exercice 1 : Addition de vecteurs** $C = A + B$ (`vecadd.c` → `vecadd.cu`) : allocation/libération de la mémoire du *device*, transferts hôte/*device*, lancement du noyau et mesure des temps. Trois configurations de lancement sont comparées : `<<<N, 1>>>`, `<<<1, N>>>` et `<<<N/T, T>>>`

Les temps GPU sont comparés au temps CPU de référence (accélération $S = T_{cpu}/T_{gpu}$) en fonction de la taille N des vecteurs.

Rappels sur le modèle d'exécution CUDA

Un programme CUDA s'exécute sur deux processeurs dotés de mémoires séparées : l'**hôte** (CPU et mémoire centrale) et le ***device*** (GPU et sa mémoire globale). Un *noyau* est lancé sur une **grille** de **blocs**, chaque bloc contenant des **threads**. Tous les threads exécutent le même code et se distinguent par leurs indices :

Notion	**Rôle**	**Valeur / limite**
`threadIdx.x`	indice du thread dans son bloc	$0 \dots T-1$
`blockIdx.x`	indice du bloc dans la grille	$0 \dots B-1$
`blockDim.x`	nombre de threads par bloc (T)	au plus 1024
`gridDim.x`	nombre de blocs (B)	jusqu'à $2^{31}-1$ en dimension x
warp	groupe de threads exécutés ensemble par un multiprocesseur	32 threads

TABLE 1 – Variables et notions utilisées pour indexer le travail

L'indice global d'un thread, utilisé par le noyau d'addition, est $i =$ `blockIdx.x` $\times$ `blockDim.x` + `threadIdx.x`.

Conséquences pour le choix de la configuration :

— Un bloc s'exécute sur **un seul** multiprocesseur ; la grille doit donc contenir assez de blocs pour occuper tous les multiprocesseurs du GPU, et chaque bloc assez de threads pour les occuper eux-mêmes.
— Les threads sont exécutés par *warps* de 32 : un bloc d'un seul thread mobilise un warp dont 31 voies sont inactives, et la taille de bloc doit être un multiple de 32 pour remplir les warps ($T = 512$ convient).
— Le lancement d'un noyau est **asynchrone** : l'hôte reprend la main avant la fin du calcul. Chronométrer le noyau avec une horloge CPU serait donc faux ; on utilise des événements CUDA suivis d'un `cudaEventSynchronize`.

Déroulement d'un programme CUDA :

1. allocation et initialisation des données sur l'hôte ;
2. allocation sur le *device* (`cudaMalloc`) ;
3. copie hôte→*device* (`cudaMemcpy`, `cudaMemcpyHostToDevice`) ;
4. lancement du noyau (`noyau<<<blocs, threads>>>(...)`) ;
5. copie *device*→hôte du résultat (`cudaMemcpyDeviceToHost`) ;
6. libération des mémoires (`cudaFree`, `free`).

Architecture du projet

Les codes fournis (`cuda_ex_0.c`, `cuda_ex_0.cu`, `vecadd.c`, `vecadd.cu`) sont conservés, et le travail est réorganisé en modules :

- `Ex0/` : `hello.cu / hello.cuh` (noyau vide et fonction `hello_cuda`), `main.cu`, `Makefile`
- `Ex1/vecteur.cu / vecteur.cuh` : noyau `kernel_vecadd`, référence CPU et les trois fonctions GPU (une par configuration)
- `Ex1/benchmark.cu / benchmark.cuh` : mesures, vérification du résultat, écriture du CSV
- `Ex1/main.cu` : lecture de N (argument), contrôle de la divisibilité par T, appel du benchmark
- `Ex1/Makefile` : compilation (`make`), mesures (`make bench`) et graphes (`make graphes`)
- `Ex1/graphes.py` : génération des 3 graphes (`matplotlib`) à partir de `resultats.csv`
- `Ex1/resultats.csv` : une ligne par taille et par configuration (colonnes `taille`, `config`, `t_cpu`, `t_gpu`)
- `Ex0/cuda_ex_0.c`, `Ex0/cuda_ex_0.cu`, `Ex1/vecadd.c`, `Ex1/vecadd.cu` : versions de l'énoncé, conservées telles quelles (`vecadd.cu` complété avec l'allocation, les transferts et le lancement)
- `bin/` : exécutables produits par les Makefiles (`ex0_hello`, `ex1_vecadd`)

T (`T_BLOCK`) est une constante de compilation, fixée à 512 par le Makefile (`-DT_BLOCK=512`). Le noyau n'ayant pas de test de borne, N doit être un multiple de T ; `main.cu` le vérifie. Les graphes sont produits avec `python3 graphes.py resultats.csv graphes`.

Protocole de mesure et validation

- **GPU utilisé :** NVIDIA GeForce RTX 3060 Laptop GPU (relevé avec `nvidia-smi`), *compute capability* **8.6** (architecture Ampere), **6 144 Mio** de mémoire, pilote **580.178.04**, version CUDA maximale supportée par le pilote : **13.0**. Compilation avec `nvcc -O2`.
- **Tailles testées :** $N \in \{512,\ 1024,\ 65\,536,\ 262\,144,\ 720\,896,\ 1\,048\,576,\ 4\,194\,304\}$, toutes multiples de $T = 512$; 720 896 est la taille par défaut du code du professeur. La configuration `<<<1, N>>>` n'est testée que pour $N \leq 1024$ (limite matérielle de 1024 threads par bloc).
- T_{cpu} : temps de la boucle séquentielle (`gettimeofday`), moyenne sur **5 répétitions**.
- T_{gpu} : temps du **noyau seul**, mesuré avec des événements CUDA (`cudaEventRecord` puis `cudaEventElapsedTime`), moyenne sur **5 répétitions**. Les `cudaMalloc` et les

`cudaMemcpy` sont **exclus** de la mesure, comme le chronomètre du code CPU ne couvre que l'appel à `vecAdd`.
— Accélération $S = T_{cpu}/T_{gpu}$ et débit MFLOPS $= N/(T \cdot 10^6)$ (une addition flottante par élément).
— Après chaque configuration, le résultat GPU est comparé élément par élément à la référence CPU (`OK`/`ECHEC`).
— Chaque répétition GPU refait le cycle complet allocation → copie vers le *device* → noyau chronométré → copie vers l'hôte → libération ; seul le noyau est chronométré.
— Les trois configurations sont mesurées dans l'ordre `<<<N,1>>>`, `<<<N/T,T>>>` puis `<<<1,N>>>`, dans un même processus par taille ; la référence CPU est mesurée avant.

Complexité théorique

Opération	**Calcul séq.**	**Calcul / thread**	**Threads utiles**	**Données échangées**	**Calcul / donnée**
Addition de vecteurs	$O(N)$	$O(1)$	N	$3N$ flottants $= 12N$ octets	$O(1)$

TABLE 2 – Complexité de l'addition de vecteurs (N : taille des vecteurs)

Chaque thread effectue une seule addition et chaque élément n'est lu qu'une fois : l'intensité arithmétique est de 1/12 flop par octet. L'opération est donc **limitée par la bande passante mémoire** et non par la puissance de calcul : le temps du noyau suit $T_{gpu} \approx T_{lancement} + 12N/B$, où B est la bande passante mémoire du *device*, et le gain maximal par rapport au CPU est de l'ordre du rapport des bandes passantes mémoire.

1 Exercice 0 : CUDA Hello World

Compilation. Un fichier C ordinaire se compile tel quel avec `nvcc`, qui lui applique simplement le compilateur hôte :

```
nvcc cuda_ex_0.c -o cuda_ex_0
```

Le programme est du C standard, sans code device :

```
#include <stdlib.h>
#include <stdio.h>
int main()
{
    printf("Hello␣World!\n");
    return EXIT_SUCCESS;
}
```

Ajout de code device. Le mot-clé `__global__` déclare une fonction exécutée sur le GPU et appelée depuis l'hôte ; l'appel avec les chevrons `<<<...>>>` est le *kernel launch*. Ici, le noyau est vide :

```
__global__ void kernel(void) { }

int main() {
    kernel<<<1, 1>>>();
    printf("Hello␣World!\n");
    return EXIT_SUCCESS;
}
```

Compilation et exécution du programme avec noyau :

```
$ nvcc cuda_ex_0.cu -o cuda_ex_0
$ ./cuda_ex_0
Hello World!
```

Version modulaire (dossier `Ex0/`). Le même programme est découpé en un en-tête, un fichier contenant le noyau et la fonction qui le lance, et un `main` :

```
/* hello.cuh */
#ifndef HELLO_CUH
#define HELLO_CUH
void hello_cuda(void);
#endif

/* hello.cu */
#include "hello.cuh"
#include <stdio.h>
__global__ void kernel(void) { }
void hello_cuda(void) {
    kernel<<<1, 1>>>();
    printf("Hello␣World!\n");
}

/* main.cu */
#include <stdlib.h>
#include "hello.cuh"
int main(void) {
```

```
    hello_cuda();
    return EXIT_SUCCESS;
}
```

Le `Makefile` compile les deux fichiers ensemble (`make`), les exécute (`make run`) et supprime `bin/` (`make clean`).

Observations :

- `nvcc` sépare le fichier en deux parties : `kernel()` est compilée pour le GPU, `main()` par le compilateur hôte. Les deux programmes (`.c` et `.cu`) affichent le même message : le `printf` est exécuté par le **CPU**, le noyau ne fait rien.
- Le lancement est **asynchrone** et le code ne vérifie aucune erreur (`cudaGetLastError`) et ne synchronise pas (`cudaDeviceSynchronize`) : l'affichage de « Hello World ! » ne prouve donc pas, à lui seul, que le noyau s'est exécuté (un lancement qui échoue serait silencieux).
- Dans notre projet, le même programme est séparé en `hello.cu` (noyau + fonction) et `main.cu`, compilés ensemble par le Makefile (`nvcc -O2 main.cu hello.cu -o bin/ex0_hello`).

2 Exercice 1 : Addition de vecteurs

2.1 Principe

Le code `vecadd.cu` reprend `vecadd.c` en ajoutant les trois étapes demandées : allocation/libération mémoire sur le *device* (`cudaMalloc`, `cudaFree`), transferts (`cudaMemcpy` hôte→*device* pour A et B, *device*→hôte pour C) et lancement du noyau, où chaque thread calcule un élément :

```
__global__ void kernel_vecadd(float *A, float *B, float *C) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    C[i] = A[i] + B[i];
}
```

Code hôte de `vecadd.cu`. Les ajouts par rapport à `vecadd.c` sont l'allocation sur le *device*, les deux copies vers le *device*, le lancement, la copie du résultat et les libérations :

```
cudaMalloc((void**)&devPtrA, N * sizeof(float));
cudaMalloc((void**)&devPtrB, N * sizeof(float));
cudaMalloc((void**)&devPtrC, N * sizeof(float));

cudaMemcpy(devPtrA, A, N * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(devPtrB, B, N * sizeof(float), cudaMemcpyHostToDevice);

vecAdd<<<N/512, 512>>>(devPtrA, devPtrB, devPtrC);

cudaMemcpy(C, devPtrC, N * sizeof(float), cudaMemcpyDeviceToHost);

cudaFree(devPtrA); cudaFree(devPtrB); cudaFree(devPtrC);
```

Dans `vecadd.cu`, la configuration `<<<N/512, 512>>>` est écrite en dur. La version modulaire (`vecteur.cu`) la paramètre pour comparer les trois lancements, et ajoute le chronométrage avec des événements CUDA. Le lancement étant asynchrone, `cudaEventSynchronize` attend la fin du noyau avant de lire le temps écoulé :

```
static double launch_and_time(float *dA, float *dB, float *dC,
                              int blocks, int threads) {
    cudaEvent_t start, stop;
    float ms;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    cudaEventRecord(start);
    kernel_vecadd<<<blocks, threads>>>(dA, dB, dC);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    cudaEventElapsedTime(&ms, start, stop);
    cudaEventDestroy(start);
    cudaEventDestroy(stop);
    return (double)ms / 1e3;      /* secondes */
}

t = launch_and_time(dA, dB, dC, N, 1);                    /* <<<N, 1>>>
     */
t = launch_and_time(dA, dB, dC, 1, N);                    /* <<<1, N>>>
     */
t = launch_and_time(dA, dB, dC, N / T_BLOCK, T_BLOCK);  /* <<<N/T, T
   >>> */
```

Le noyau est identique pour les trois configurations ; seul le lancement change :

Lancement	Blocs	Threads / bloc	Remarque
`<<<N, 1>>>`	N	1	un thread par bloc : le parallélisme à l'intérieur des blocs n'est pas exploité
`<<<1, N>>>`	1	N	un seul bloc, donc un seul multiprocesseur ; $N \leq 1024$
`<<<N/T, T>>>`	N/T	$T = 512$	compromis : blocs nombreux et remplis de threads

TABLE 3 – Les trois configurations de lancement comparées

Déroulement d'une mesure (`benchmark.cu`) :

1. génération de A et B avec les formules de `vecadd.c` (`A[i] = (float)random()`, `B[i] = (float)RAND_MAX - A[i]`) ;
2. référence CPU : 5 exécutions de la boucle d'addition, moyenne des temps ;
3. pour chaque configuration : 5 cycles complets (allocation, copie, noyau chronométré, copie du résultat, libération), moyenne des temps du noyau ;
4. comparaison du résultat GPU à la référence CPU, affichage de T_{cpu}, T_{gpu}, S, MFLOPS et `OK`/`ECHEC`, puis ajout au fichier CSV.

Compilation et exécution :

```
make          # compile bin/ex1_vecadd  (nvcc -O2 -DT_BLOCK=512)
make bench    # une execution par taille -> resultats.csv
```

```
make graphes  # graphes/vecadd_temps.png, _speedup.png, _mflops.png
bin/ex1_vecadd 1048576 resultats.csv     # une seule taille
```

2.2 Résultats

N	T_{cpu} (μs)	`<<<N,1>>>` : T_{gpu} (μs) [S]	`<<<1,N>>>` : T_{gpu} (μs) [S]	`<<<N/T,T>>>` : T_{gpu} (μs) [S]
512	0,4	3338,9 [$1{,}2 \cdot 10^{-4}$]	7,2 [0,06]	6,6 [0,06]
1024	0,8	34,8 [0,02]	9,0 [0,09]	5,9 [0,14]
65 536	37,4	218,9 [0,17]	—	11,1 [3,4]
262 144	341,6	511,9 [0,67]	—	64,0 [5,3]
720 896	619,0	1127,9 [0,55]	—	39,7 [15,6]
1 048 576	774,0	1595,3 [0,49]	—	58,6 [13,2]
4 194 304	5728,8	6307,4 [0,91]	—	212,8 [26,9]

Table 4 – Temps moyens (5 répétitions, noyau seul) et accélération $S = T_{cpu}/T_{gpu}$

N	CPU	`N,1`	`1,N`	`N/T,T`	Blocs (N/T)	Débit mémoire (Go/s)	Volume $12N$ (Mo)
512	1 280	0,15	71	78	1	0,9	0,006
1024	1 280	29	114	172	2	2,1	0,012
65 536	1 752	299	—	5 916	128	71,0	0,79
262 144	767	512	—	4 097	512	49,2	3,1
720 896	1 165	639	—	18 170	1 408	218,0	8,7
1 048 576	1 355	657	—	17 898	2 048	214,8	12,6
4 194 304	732	665	—	19 714	8 192	236,6	50,3

Table 5 – Débit en MFLOPS (CPU et GPU ; `N,1`, `1,N` et `N/T,T` désignent les trois lancements), débit mémoire effectif de `<<<N/T,T>>>` (calculé avec $12N$ octets par noyau) et volume de données

N	Rapport $T_{N,1}/T_{N/T,T}$	`N,1` : ns par bloc	`N/T,T` : ns par élément	CPU : ns par élément
512	509	6521	12,8	0,78
1024	5,9	34,0	5,8	0,78
65 536	19,8	3,34	0,169	0,57
262 144	8,0	1,95	0,244	1,30
720 896	28,4	1,57	0,055	0,86
1 048 576	27,2	1,52	0,056	0,74
4 194 304	29,6	1,50	0,051	1,37

Table 6 – Coût comparé des configurations (temps divisé par le nombre de blocs ou d'éléments ; $T_{N,1}$ et $T_{N/T,T}$: temps GPU des lancements `<<<N,1>>>` et `<<<N/T,T>>>`)

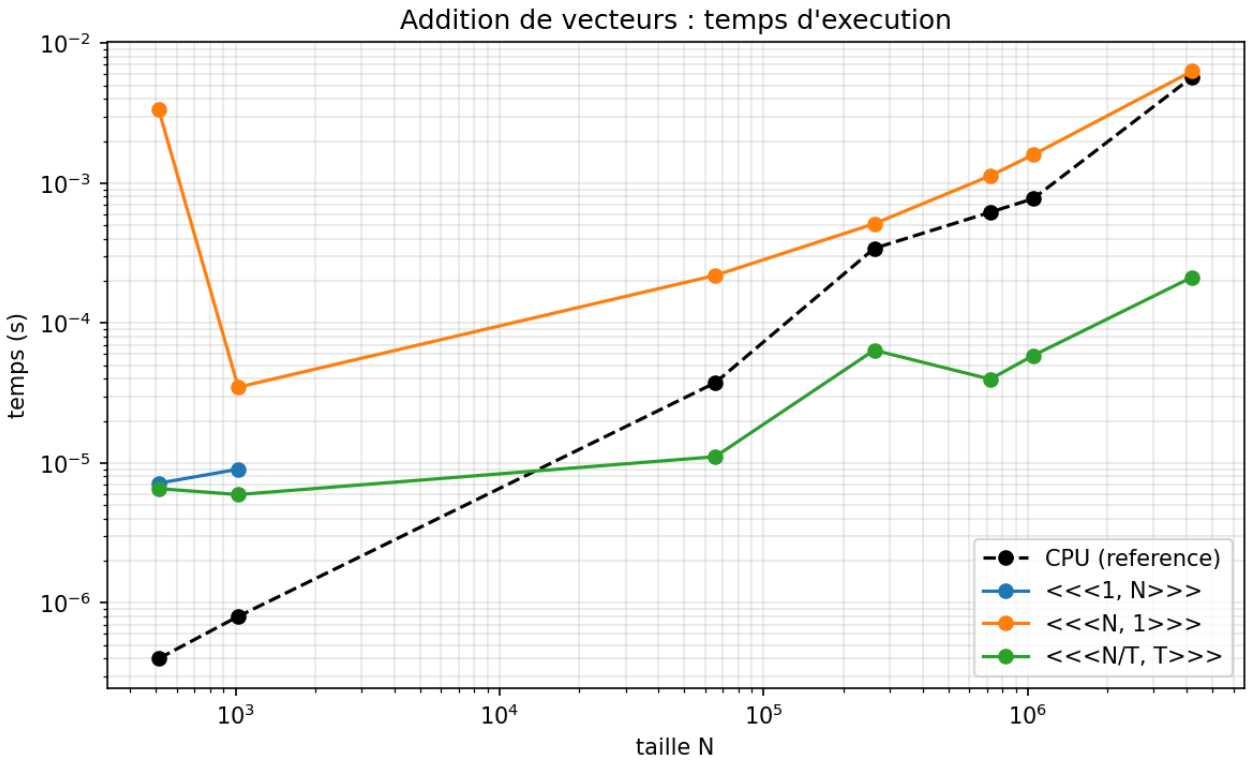


FIGURE 1 – Temps d'exécution

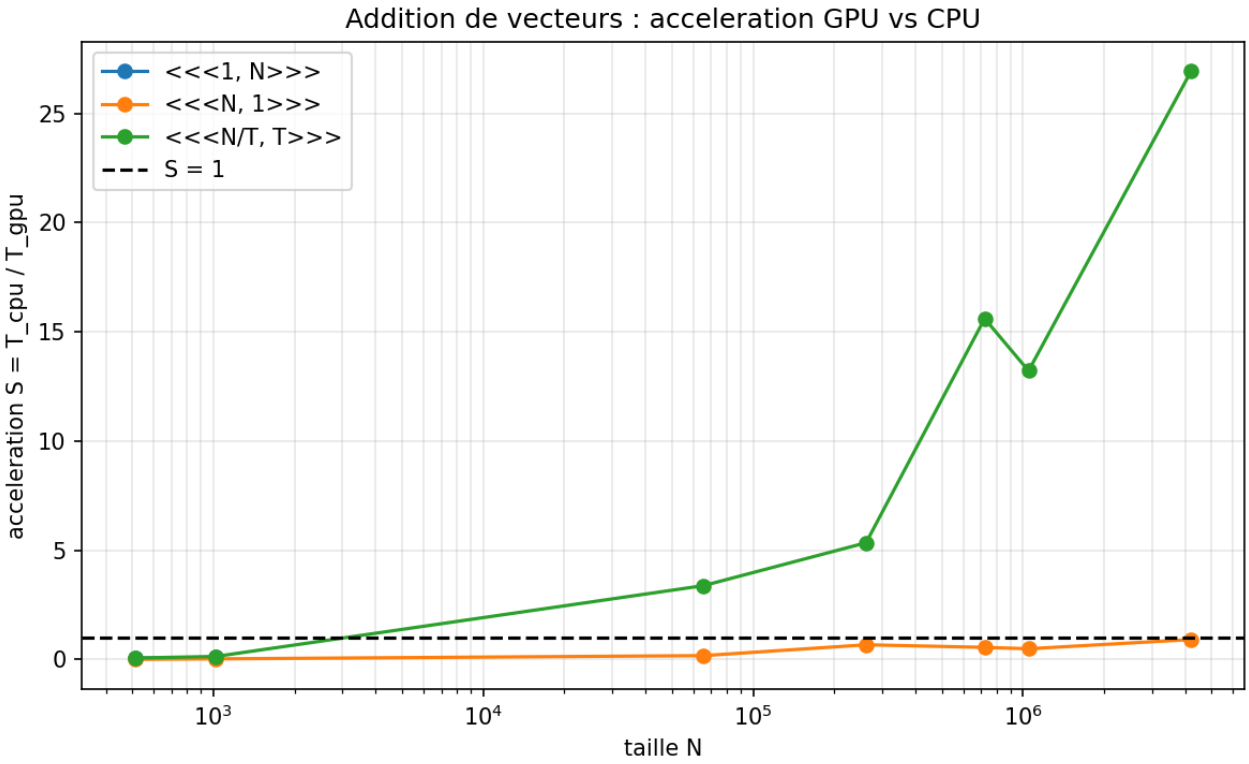


FIGURE 2 – Accélération GPU vs CPU

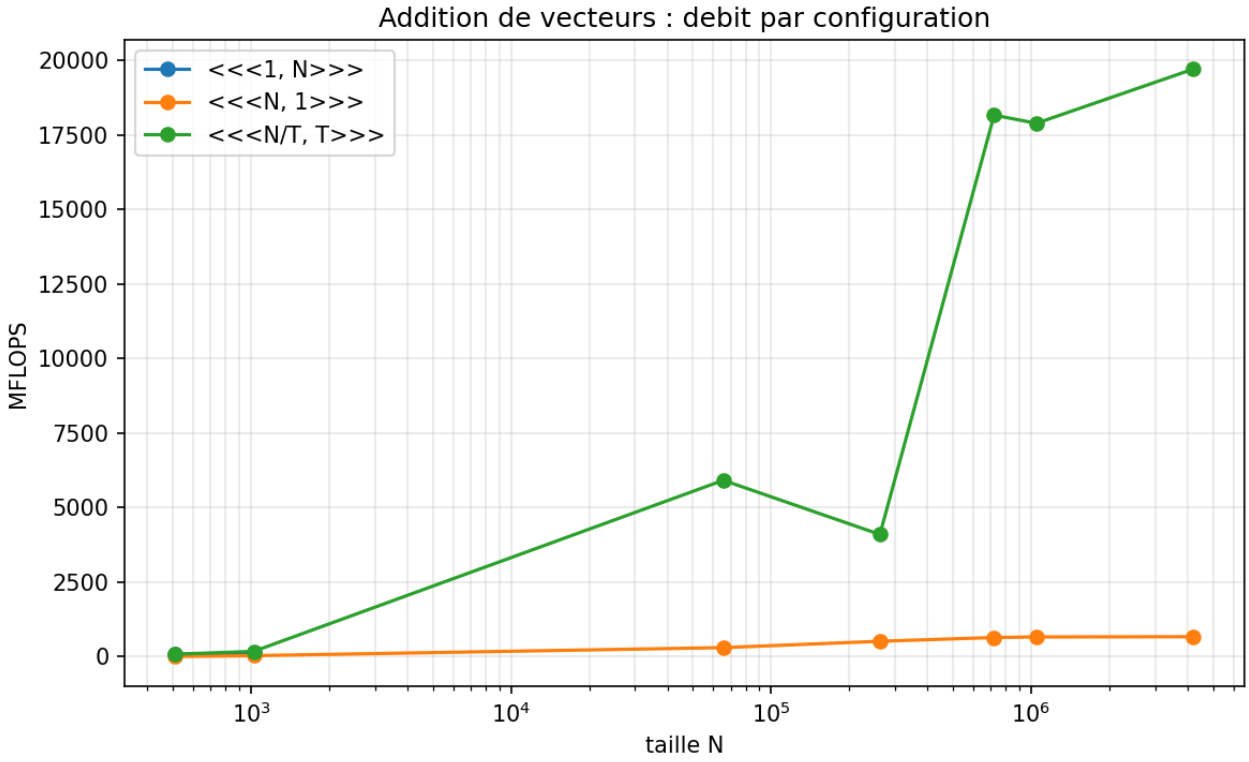


FIGURE 3 – Débit (MFLOPS)

Observations :

— `<<<N, 1>>>` **est la pire configuration** pour toutes les tailles : elle est de 8 à 30 fois plus lente que `<<<N/T, T>>>` dès $N \geq 65\,536$ (par exemple 6,3 ms contre 0,21 ms pour $N = 4\,194\,304$). Son accélération reste inférieure à 1 pour toutes les tailles (au mieux 0,91), c'est-à-dire que le GPU est **moins rapide que le CPU** (figure 2).

— Son débit plafonne vers 500 à 665 MFLOPS dès $N \geq 262\,144$ (figure 3) : le temps est proportionnel au nombre de blocs, ce qui est la signature d'un goulot d'ordonnancement. Chaque bloc d'un seul thread n'occupe qu'une voie sur 32 de son *warp* et il faut créer et distribuer N blocs.

— `<<<N/T, T>>>` **devient rentable entre** $N = 10^3$ **et** $N = 6{,}5 \cdot 10^4$: l'accélération passe de 0,14 à 3,4, puis atteint 26,9 pour $N = 4\,194\,304$. Le débit monte à environ 18–20 GFLOPS dès $N \geq 720\,896$.

— Ce débit correspond à environ 215–237 Go/s de mémoire échangée (calcul : 12 octets par élément, soit 2 lectures et 1 écriture de 4 octets), à comparer à la bande passante crête de la carte (NVIDIA GeForce RTX 3060 Laptop GPU, 6 Go) : l'addition de vecteurs est **limitée par la mémoire** et non par le calcul (une seule opération par 12 octets).

— Pour $N \leq 1024$, le GPU est toujours plus lent que le CPU ($S \leq 0{,}14$) : T_{gpu} reste de l'ordre de 6 à 9 μs quelle que soit la configuration, alors que le CPU n'a besoin que de 0,4 à 0,8 μs. Ce plancher correspond au coût de lancement du noyau. Le T_{cpu} de ces deux tailles est de plus à la limite de la résolution de `gettimeofday` (la microseconde), donc imprécis.

— `<<<1, N>>>` n'est possible que pour $N \leq 1024$ et utilise un seul multiprocesseur. Pour $N = 512$, `<<<N/T, T>>>` est *exactement* le lancement `<<<1, 512>>>` : l'écart mesuré (6,6 contre 7,2 μs) est du bruit de mesure. Pour $N = 1024$, `<<<N/T, T>>>` lance 2 blocs de 512 threads et est un peu plus rapide (5,9 contre 9,0 μs). Cette configuration ne passe pas à l'échelle puisqu'elle plafonne à 1024 éléments.

— **Points irréguliers.** Le point `<<<N,1>>>` à $N = 512$ (3,3 ms) est aberrant : il est attribué au coût d'initialisation du premier lancement dans le processus, car cette configuration est mesurée en premier et la moyenne porte sur 5 répétitions seulement. Le recul de l'accélération entre $N = 720\,896$ (15,6) et $N = 1\,048\,576$ (13,2) vient à la fois du CPU (de 1165 à 1355 MFLOPS) et du GPU (39,7 puis 58,6 μs) : c'est de la variabilité de mesure.

2.3 Analyse complémentaire

Pourquoi `<<<N, 1>>>` est-elle si lente ?

— Chaque bloc ne contient qu'un thread : un *warp* n'a qu'une voie active sur 32 (soit 3 % des unités d'exécution utilisées).
— Un multiprocesseur n'héberge qu'un nombre limité de blocs à la fois (16 pour la *compute capability* 8.6 de la génération Ampere, contre 1536 threads résidents au maximum, d'après la documentation NVIDIA). Avec des blocs d'un thread, seuls 16 threads par multiprocesseur sont en vol, ce qui ne permet pas de masquer la latence mémoire.
— Le temps par bloc se stabilise à $\approx 1{,}5$ ns pour $N \geq 7 \cdot 10^5$ (tableau 6), soit ≈ 665 millions de blocs par seconde : la durée est proportionnelle au **nombre de blocs** et indépendante du travail utile, ce qui indique que le goulot est l'ordonnancement des blocs.
— Pour $N \geq 65\,536$, `<<<N/T, T>>>` est de 8 à 30 fois plus rapide, avec le même noyau et le même nombre de threads au total.

Influence de la taille N (`<<<N/T, T>>>`) : trois régimes apparaissent.

— *Régime de latence* ($N \leq 1024$, 1 à 2 blocs) : $T_{gpu} \approx 6$ μs, coût fixe de lancement. Le CPU termine en moins d'une microseconde, aucun gain n'est possible.
— *Transition* ($65\,536 \leq N \leq 262\,144$, 128 à 512 blocs) : le GPU est 3,4 à 5,3 fois plus rapide, mais le débit mémoire (49 à 71 Go/s) reste loin de son maximum : pour $N = 65\,536$, plus de la moitié des 11 μs correspond encore au coût fixe de lancement.
— *Saturation* ($N \geq 720\,896$, au moins 1408 blocs) : le débit mémoire se stabilise à 215–237 Go/s et T_{gpu} devient proportionnel à N (de 720 896 à 4 194 304, les données sont multipliées par 5,8 et le temps par 5,4). L'accélération continue pourtant de croître (15,6 puis 26,9) parce que le coût par élément du CPU augmente (0,86 puis 1,37 ns, tableau 6) lorsque les trois tableaux, de 48 Mo au total pour $N = 4\,194\,304$, ne tiennent plus dans les caches.

Passage à l'échelle. Sur $65\,536 \leq N \leq 4\,194\,304$, la pente en échelle log-log du temps vaut $\approx 1{,}14$ pour le CPU (théorie : 1, l'écart venant des effets de cache), $\approx 0{,}80$ pour `<<<N, 1>>>` et $\approx 0{,}62$ pour `<<<N/T, T>>>` : sur cette plage, le temps GPU croît moins vite que N parce que le coût fixe de lancement est progressivement amorti.

Place des transferts. L'addition échange $12N$ octets entre l'hôte et le *device* (tableau 5), soit 50,3 Mo pour $N = 4\,194\,304$, alors que le CPU effectue l'addition en 5,7 ms. Si le lien PCIe soutenait de l'ordre de 10 Go/s (ordre de grandeur supposé, non mesuré), ces seuls transferts dureraient ≈ 5 ms, soit plus de 20 fois le temps du noyau (212,8 μs) : une version GPU complète (allocation, transferts, noyau) ne serait alors pas plus rapide que le CPU. L'accélération de 26,9 est celle du **noyau seul**. Une opération à faible intensité arithmétique (1/12 flop par octet) ne justifie le GPU que si les données y restent entre plusieurs noyaux ; c'est la raison pour laquelle les opérations du Lab 2 (réduction, produit matriciel en $O(N^3)$ calculs pour $O(N^2)$ données) sont de meilleurs candidats.

Fiabilité des mesures.

— Les temps ne sont pas monotones : $T_{gpu} = 64$ μs pour $N = 262\,144$ contre 39,7 μs pour $N = 720\,896$, 2,75 fois plus grand. Le CPU présente la même anomalie (767 contre 1 165 MFLOPS). Les points intermédiaires, entre $2 \cdot 10^5$ et 10^6, sont donc à interpréter avec prudence ; la tendance (accélération croissante avec N) reste nette.
— Le point aberrant `<<<N, 1>>>` à $N = 512$ (3,3 ms) est attribué à l'initialisation du premier lancement du processus ; le GPU étant au repos (état de performance P3 relevé par `nvidia-smi`), une montée en fréquence peut aussi y contribuer. Ces deux explications restent des hypothèses.
— Pour $N \leq 1024$, T_{cpu} (0,4 et 0,8 μs) est proche de la résolution de `gettimeofday` (la microseconde) : les accélérations correspondantes sont indicatives.

3 Synthèse et interprétation

Configuration	**Parallélisme exploité**	**Accélération max observée**	**Verdict**
`<<<N, 1>>>`	N blocs d'un seul thread (1 voie active sur 32 par warp)	0,91 ($N = 4\,194\,304$)	Jamais rentable ; le coût d'ordonnancement des blocs domine
`<<<1, N>>>`	1 bloc de N threads, un seul multiprocesseur	0,09 ($N = 1024$)	Jamais rentable ; limitée à $N \leq 1024$
`<<<N/T, T>>>`	N/T blocs de 512 threads	26,9 ($N = 4\,194\,304$)	Rentable dès $N \approx 6{,}5 \cdot 10^4$; de mieux en mieux avec N

TABLE 7 – Synthèse des trois configurations (valeurs relevées dans les tableaux)

Le modèle $T_{gpu} \approx T_{lancement} + 12N/B$ explique l'ensemble des résultats :

— **Coût fixe** : $T_{lancement} \approx 6$ μs, qui explique pourquoi le GPU perd contre le CPU pour $N \leq 1024$.
— **Terme proportionnel à** N : pour $N \geq 7 \cdot 10^5$, environ 0,05 ns par élément, soit $\approx$ 215–237 Go/s ; c'est la bande passante mémoire qui limite, comme le laisse prévoir l'intensité arithmétique de 1/12 flop par octet.
— **Configuration de lancement** : seule `<<<N/T, T>>>` fournit assez de blocs pour occuper tous les multiprocesseurs et assez de threads par bloc pour remplir les warps ; les deux autres configurations ne peuvent pas atteindre ce régime.

Conclusion

Ce lab a permis de compiler un premier programme CUDA avec `nvcc` (noyau vide, lancement `<<<1, 1>>>`) puis de porter l'addition de vecteurs sur GPU, avec allocation, transferts et lancement du noyau, et de comparer trois configurations de grille.

Résultats clés :

- La **configuration de lancement domine la performance** : à $N = 4\,194\,304$, `<<<N/T, T>>>` est ≈ 30 fois plus rapide que `<<<N, 1>>>`, qui n'exploite pas le parallélisme intra-bloc et ne dépasse jamais le CPU. `<<<1, N>>>` est limitée à 1024 éléments et à un seul multiprocesseur.
- Avec `<<<N/T, T>>>` ($T = 512$), le GPU dépasse le CPU dès que N est assez grand (accélération $\approx 3{,}4$ pour $N = 65\,536$, $\approx 26{,}9$ pour $N = 4\,194\,304$), mais est plus lent pour $N \leq 1024$, où le coût de lancement (≈ 6 μs) l'emporte.
- L'opération est limitée par la bande passante mémoire (environ 215 à 237 Go/s observés pour $N \geq 720\,896$).
- Avec `<<<N, 1>>>`, le temps est proportionnel au nombre de blocs ($\approx 1{,}5$ ns par bloc) : la configuration, et non la quantité de calcul, fixe le temps d'exécution.
- Les accélérations sont celles du noyau seul ; avec les transferts ($12N$ octets), le gain disparaîtrait vraisemblablement pour cette opération (voir l'analyse complémentaire).

Limites et pistes : les accélérations portent sur le **noyau seul** : les transferts hôte/*device* ($\approx 3N$ floats échangés par l'addition) et les allocations sont exclus et réduiraient l'accélération réelle d'une application complète. Les mesures reposent sur 5 répétitions, sans lancement d'échauffement ; il serait préférable d'ajouter un lancement à blanc avant la mesure, d'utiliser la médiane, de tester d'autres valeurs de T (128, 256, 1024) et de mesurer les transferts séparément.